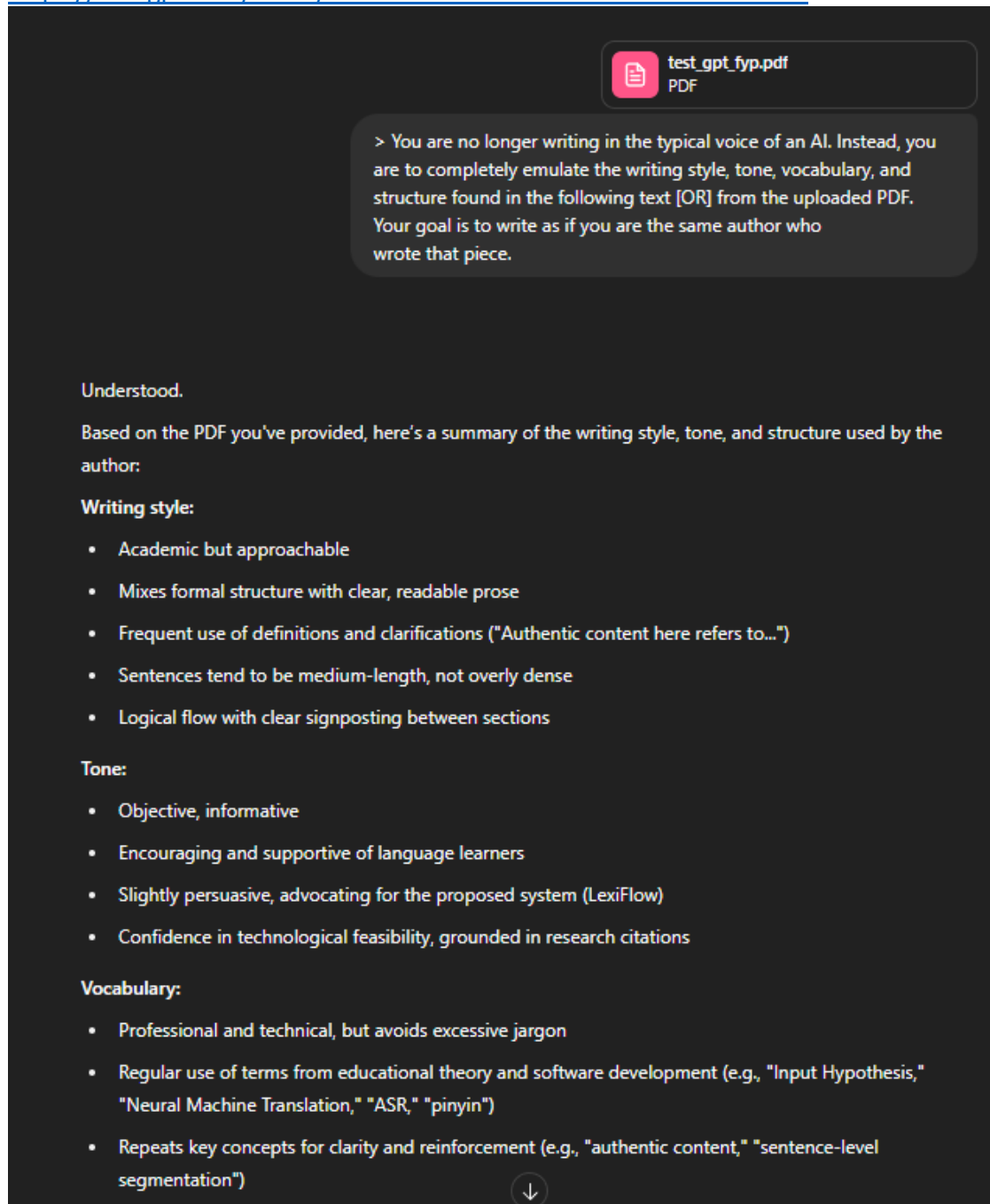


Appendix 1:

List of Prompts Used

1. <https://chatgpt.com/share/6874acf9-d20c-8013-9e8a-1874ae62b27c>



The screenshot shows a ChatGPT chat window with a dark background. At the top right, a pink icon of a document with a checkmark is next to the text "test_gpt_fyp.pdf" and "PDF". Below this, a grey speech bubble contains the instruction: "> You are no longer writing in the typical voice of an AI. Instead, you are to completely emulate the writing style, tone, vocabulary, and structure found in the following text [OR] from the uploaded PDF. Your goal is to write as if you are the same author who wrote that piece." The main chat area contains the following text: "Understood." followed by "Based on the PDF you've provided, here's a summary of the writing style, tone, and structure used by the author:". This is followed by three sections: "Writing style:" with a bulleted list of five points; "Tone:" with a bulleted list of four points; and "Vocabulary:" with a bulleted list of three points. At the bottom center, there is a small circular button with a downward arrow.

test_gpt_fyp.pdf
PDF

> You are no longer writing in the typical voice of an AI. Instead, you are to completely emulate the writing style, tone, vocabulary, and structure found in the following text [OR] from the uploaded PDF. Your goal is to write as if you are the same author who wrote that piece.

Understood.

Based on the PDF you've provided, here's a summary of the writing style, tone, and structure used by the author:

Writing style:

- Academic but approachable
- Mixes formal structure with clear, readable prose
- Frequent use of definitions and clarifications ("Authentic content here refers to...")
- Sentences tend to be medium-length, not overly dense
- Logical flow with clear signposting between sections

Tone:

- Objective, informative
- Encouraging and supportive of language learners
- Slightly persuasive, advocating for the proposed system (LexiFlow)
- Confidence in technological feasibility, grounded in research citations

Vocabulary:

- Professional and technical, but avoids excessive jargon
- Regular use of terms from educational theory and software development (e.g., "Input Hypothesis," "Neural Machine Translation," "ASR," "pinyin")
- Repeats key concepts for clarity and reinforcement (e.g., "authentic content," "sentence-level segmentation")

2. <https://chatgpt.com/share/6874ad62-f5ac-8013-9136-c53b753af556>

for my use case diagram my supervisor asked me to add my models into the diagram. this was her comment: "if you have components which you add as API put it as an actor

ASR - OpenAI Whisper?

ML - Seamless M4T?

"

under what stereotype do i add them ? external systems? api? models?

for reference both models will run locally on my system so no external api will be used

Great question — and you're already on the right track by thinking in terms of **stereotypes** and **actors** in a UML Use Case Diagram.

Your supervisor's suggestion:

"If you have components which you add as API, put it as an actor."

Even though the models (e.g., OpenAI Whisper for ASR and SeamlessM4T for ML translation) run **locally**, your system **depends** on them as **external functional components**. In UML terms, that makes them actors — not because they're human users, but because they're **external to your system's logic**, even if they're hosted locally.

So what stereotype should you use?

✅ Recommended stereotype:

plaintext

📋 Copy ✎ Edit

<<external system>>

This is the most common and UML-appropriate stereotype for models/APIs/tools that interact with your system but are not part of its internal logic, even if they run locally.



You could label them like:

Additional Prompts Used

(AI-Assisted Development)

A.1 Reviewing FYP2 grading rubric alignment

Asked the assistant to compare the implemented system and thesis report against the official FYP2 grading rubric, surfacing missing deliverables such as the UAT Report and Test Execution Report.

compare the @thesis-template.pdf with the @CQI SECJ Rubric FYP 2 - Development.xlsx 11 sep 2024 8am.pdf i need to submit soon so let me know what is missing from the system and report.

compare the code in details with the rebric and compare it with the report. make sure everything in the system in mention in report and they align with what the rubric wants

1h ago

Ran 2 commands, read 2 files, ran 2 agents >

I now have the full picture: the rubric (PSM2/FYP2, the *final* one), the actual report content (thesis-template.pdf), and the actual codebase state. Here's the comparison.

 Critical Issue first

The rubric you attached is FYP2 (PSM2) — the final submission rubric (Output 40%, Final Report 20%, Software Documents 10%, Supervision 10%, Presentation 15%, Short Paper 5%). But your report's Chapter 6 Conclusion explicitly says:

"[Testing/evaluation] was going to be done... in the next phase which is PSM2... This objective will be considered complete once the project is fully developed and testing is conducted."

So the report you have is written as if it's still the PSM1 report, but you're being graded against the PSM2 rubric. If this is your final submission, this paragraph alone will read as "testing was never done" to evaluators — even though your code is actually far more complete than the report admits.

What the rubric requires that's missing from the report/docs

Rubric item	Weight	Status
UAT Report	5% (Software Docs)	✗ Not in report. Appendix C has informal stakeholder feedback (Malay, 1 meeting) — not a formal UAT report with test cases/criteria.
Test Execution Report	5% (Software Docs)	✗ Missing entirely. No unit/integration test logs — codebase confirms "test": "echo \"Error: no test specified\"" in package.json.
Implementation and Testing (Final Report criterion, 15%)	15%	⚠ Implementation is described well (Ch5), but test cases, coverage %, debugging records are absent.

lexiflow main

+27,684 -2,803

Create PR

A.2 Requesting a full frontend redesign for reviewer clarity

Instructed the assistant (via the web-artifacts-builder skill) to rebuild the entire frontend because it was lacking, confusing, and not user-friendly, with an explicit requirement to add info pop-ups explaining unfamiliar concepts for a lecturer reviewing the project, and to apply one consistent theme across the system.

use web-artifacts-builder to go through the whole project and redo the frontend. it was described as lacking and confusing and distracting and inconsistent and not user friendly. for the new system make sure that the system is incredibly user friendly and easy to use. and i info popups to explain any new concept that a lecturer reviewing and marking this project might need to understand or use it. focus on clean and easy to use and full of info. create a theme and use it in the whole system.

I'll start by invoking the web-artifacts-builder skill and exploring the project structure in parallel.

Ran 2 commands, used a tool >

This is a substantial task. Let me deeply understand the existing frontend before rebuilding it. Let me explore the frontend source.

Ran 2 commands >

Let me read the core views to understand all features.

Ran 3 commands >

Let me understand the backend API and the data models, plus the view logic (script sections).

Ran 2 commands >

I now understand the system well. Let me quickly capture the remaining feature logic from the three large views (Watch, List, Flashcard), then start building.

Ran 2 commands >

Let me check the subtitle data shape and the dictionary/character components to render the Watch view authentically.

Ran Inspect subtitle JSON shape and CharacterDisplay >

I have a complete picture of LexiFlow. Let me set up task tracking and initialize the artifact project.

Ran a command, used a tool >

Let me initialize the project and set up tasks.

Ran 2 commands, used 4 tools >



A.3 Requesting a feature-parity migration plan

Instructed the assistant to read every view in the legacy Vue frontend, catalogue all existing features, and produce a self-contained TODO plan file that a separate chat session could later execute against the new React frontend.

alot of functionalities are missing. go back to the old frontend. read the the views and list down all the features they provided. create a todo plan for the next chat to implement. JUST CREATE A PLAN THAT I CAN GIVE TO THE NEXT CHAT

🗨️ ↶️ ↷️ 15h ago

I'll read through the old Vue frontend systematically to catalog every feature, then write a plan file. Let me start by surveying all the views and components.

Ran Survey old Vue views feature surface >

Let me extract the deeper feature details — function/computed names and notable UI labels across the views and components.

Ran Extract function names from old views >

Let me read the notable components and grab UI labels for full fidelity.

Ran Read DictionarySearch and SubBox components >

Let me capture the remaining UI specifics — visible labels and template features in the big views.

Ran 2 commands >

I now have a complete inventory. Let me write the plan file the next chat can execute from.

Created FRONTEND_FEATURE_PLAN.md +217 -0 >

Done. I read through all 8 old Vue views and their components (`frontend-vue-legacy/`), cataloged every feature, compared against the current React app, and wrote a complete plan to:

[FRONTEND_FEATURE_PLAN.md](#) (repo root)

It's self-contained so you can hand it straight to the next chat. What's inside:

A.4 Reviewing the generated migration plan

The assistant's response surveying all legacy Vue views/components and compiling the cross-referenced feature plan (FRONTEND_FEATURE_PLAN.md) used to guide the React rewrite.

follow the @FRONTEND_FEATURE_PLAN.md. implement them cleanly and follow system theme. refer back to old frontend any time u need ideas how to implement. make sure all backend api are used by end. if something is not used you can ask how to use it.

I'll start by exploring the current frontend structure and the legacy Vue app to understand the codebase before implementing the plan.

Ran 2 agents, read 7 files, used 12 tools >

Now let's read the remaining files I need before writing code: VocabView, FlashcardsView, LibraryView, RequestView, settings.ts, concepts.ts.

Read 4 files >

Now let me check concepts.ts and the nav module quickly, then start implementing.

Read 2 files >

Now I have enough context to implement. Let's start with the API client additions (Task #1).

Edited a file, used a tool >

Now let's add the actual API methods. Need search response shape — let me check it quickly.

Ran agent Haiku 4.5 Check search and dictionary response field names >

Now I have exact shapes. Let me add the API methods.

Edited `api.ts` +24 -2 >

Now add progress methods and deck layout/reset methods.

Edited `api.ts` +33 -0 >

Also need `syncDeck` to accept front/back config for P0.2. Let me update it.

Edited `api.ts` +9 -2 >

API client done. Marking complete and moving to WatchView (P0.1, P1.4, P2.7, P2.8 all live in WatchView — I'll do them together since they touch the same file).

A.5 Rubric gap analysis - Software Documents criteria

Continuation of the rubric comparison, focusing specifically on the Software Documents (UAT Report / Test Execution Report) marking criteria and their current status.

compare the @thesis-template.pdf with the @CQI SECJ Rubric FYP 2 - Development.xlsx 11 sep 2024 8am.pdf i need to submit soon so let me know what is missing from the system and report.

compare the code in details with the rebric and compare it with the report. make sure everything in the system in mention in report and they align with what the rubric wants

Ran 2 commands, read 2 files, ran 2 agents >

I now have the full picture: the rubric (PSM2/FYP2, the *final* one), the actual report content (thesis-template.pdf), and the actual codebase state. Here's the comparison.

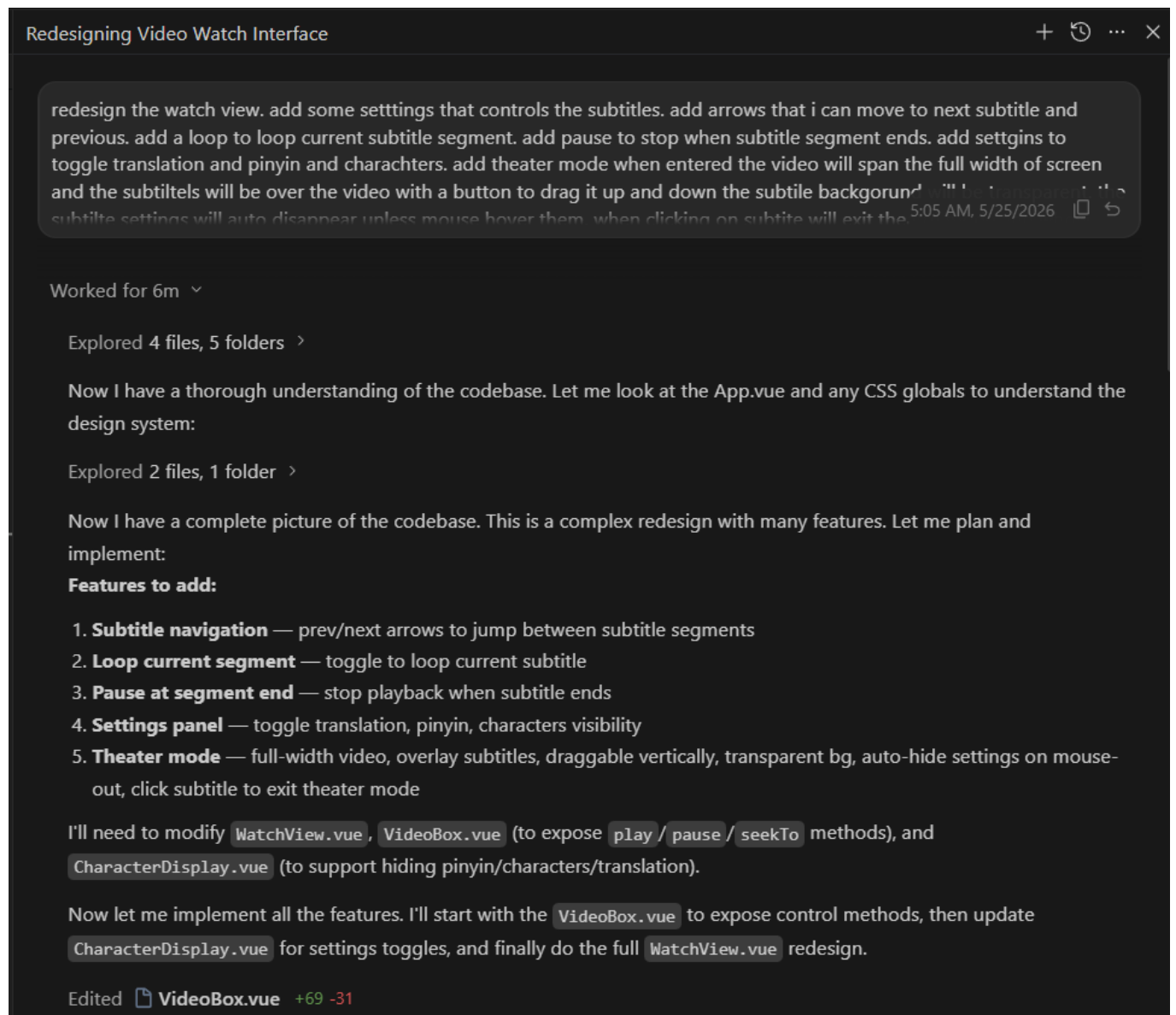
Critical Issue first

The rubric you attached is FYP2 (PSM2) — the final submission rubric (Output 40%, Final Report 20%, Software Documents 10%, Supervision 10%, Presentation 15%, Short Paper 5%). But your report's Chapter 6 Conclusion explicitly says:
"[Testing/evaluation] was going to be done... in the next phase which is PSM2... This objective will be considered complete once the project is fully developed and testing is conducted."
So the report you have is written as if it's still the PSM1 report, but you're being graded against the PSM2 rubric. If this is your final submission, this paragraph alone will read as "testing was never done" to evaluators — even though your code is actually far more complete than the report admits.
What the rubric requires that's missing from the report/docs

Rubric Item	Weight	Status
UAT Report	5% (Software Docs)	✗ Not in report. Appendix C has informal stakeholder feedback (Malay, 1 meeting) — not a formal UAT report with test cases/criteria.
Test Execution Report	5% (Software Docs)	✗ Missing entirely. No unit/integration test logs — codebase confirms "test": "echo \"Error: no test specified\"" in package.json.

A.6 Requesting Watch View redesign with theater mode

Asked the assistant to redesign the video Watch View: subtitle navigation arrows, segment looping, pause-at-segment-end, a settings panel to toggle translation/pinyin/characters, and a theater mode with a draggable, auto-hiding subtitle overlay.



A.7 Refactoring backend to Domain-Driven Modular architecture

Instructed the assistant to migrate the Node.js Express backend from a flat layered architecture (controllers/, services/, utils/) into a Domain-Driven Modular architecture, encapsulating each feature inside its own modules/ directory.

Refactoring Backend Domain Architecture

We are refactoring our Node.js Express backend from a flat layered architecture (controllers/, services/, utils/) to a Domain-Driven Modular architecture. All features must be encapsulated within a modules/ directory.

Please execute the following migration steps exactly as specified.

3:00 AM, 5/23/2026

Worked for 1m

I have analyzed the codebase and successfully created the implementation plan to migrate your Express backend to a Domain-Driven Modular architecture inside a new modules/ directory.

Please review the complete plan in the implementation_plan.md artifact.

Once you are satisfied with the proposed changes, please provide your approval so we can proceed with Phase 1 (Directory Scaffolding) and Phase 2 (Refactoring Pattern Implementation).

Implementation Plan

Implementation plan for refactoring the Node.js Express backend from a flat layered architecture to a Domain-Driven Modular architecture under modules/. This covers Phase 1 (Directory Scaffolding) and Phase 2 (Refactoring Pattern Implementation) for vocabulary, flashcards, quizzes, users, and videos (library/jobs) domains.

3:00 AM, 5/23/2026

Proceeded with Implementation Plan

Worked for 12m

33 files changed +728 -63

Review

A.8 Adding PostgreSQL to the Docker deployment pipeline

Asked the assistant how to add a PostgreSQL service to the existing Docker Compose / GitHub Actions deployment pipeline on the Tencent Cloud Lighthouse server.

i want to add postgres sql to my deployment server. for now i have github actoins that uses docker to build adn deploy.

ubuntu@VM-0-7-ubuntu:~/lexiflow\$ cat docker-compose 



I highly recommend adding PostgreSQL to Docker via your `docker-compose.yml` rather than installing it natively on the Ubuntu host.

Since you are already using Docker Compose, adding the database as another container keeps your entire infrastructure neatly bundled. It isolates the database, prevents conflicts with host packages, and makes it incredibly easy to migrate to a new server in the future if you ever outgrow your 40GB drive.

Here is how you can update your setup to include PostgreSQL.

1. Update `docker-compose.yml`

You will need to add a `db` service. Crucially, you must also add a **Docker Volume** to ensure your database data isn't wiped out every time you update your backend container.

YAML  

```
version: '3.8'

services:
  backend:
    image: ghcr.io/amer-adam/backend-node:latest
    container_name: lexiflow-express-app
    restart: always
    ports:
      - "4556:4556"
    depends_on:
      - db
```